### 7.1.3 Sushi bar non-solution

The problem is at Line 7. If a customer arrives while the bar is full, he has to give up the mutex while he waits so that other customers can leave. When the last customer leaves, she signals `block`, which wakes up at least some of the waiting customers, and clears `must_wait`.

But when the customers wake up, they have to get the mutex back, and that means they have to compete with incoming new threads. If new threads arrive and get the mutex first, they could take all the seats before the waiting threads. This is not just a question of injustice; it is possible for more than 5 threads to be in the critical section concurrently, which violates the synchronization constraints.

Reek provides two solutions to this problem, which appear in the next two sections.

Puzzle: see if you can come up with two different correct solutions!

Hint: neither solution uses any additional variables.

### 7.1.4 Sushi bar solution #1

The only reason a waiting customer has to reacquire the mutex is to update the state of `eating` and `waiting`, so one way to solve the problem is to make the departing customer, who already has the mutex, do the updating.

Sushi bar solution #1

```
mutex.wait()
if must_wait:
    waiting += 1
    mutex.signal()
    block.wait()
else:
    eating += 1
    must_wait = (eating == 5)
    mutex.signal()

# eat sushi

mutex.wait()
eating -= 1
if eating == 0:
    n = min(5, waiting)
    waiting -= n
    eating += n
    must_wait = (eating == 5)
    block.signal(n)
mutex.signal()
```

When the last departing customer releases the mutex, `eating` has already been updated, so newly arriving customers see the right state and block if necessary. Reek calls this pattern "I'll do it for you," because the departing thread is doing work that seems, logically, to belong to the waiting threads.

A drawback of this approach is that it is a little more difficult to confirm that the state is being updated correctly.

### 7.1.5 Sushi bar solution #2

Reek's alternative solution is based on the counterintuitive notion that we can transfer a mutex from one thread to another! In other words, one thread can acquire a lock and then another thread can release it. As long as both threads understand that the lock has been transferred, there is nothing wrong with this.

Sushi bar solution #2

```
mutex.wait()
if must_wait:
    waiting += 1
    mutex.signal()
    block.wait()     # when we resume, we have the mutex
    waiting -= 1

eating += 1
must_wait = (eating == 5)
if waiting and not must_wait:
    block.signal()            # and pass the mutex
else:
    mutex.signal()

# eat sushi

mutex.wait()
eating -= 1
if eating == 0: must_wait = False

if waiting and not must_wait:
    block.signal()            # and pass the mutex
else:
    mutex.signal()
```

If there are fewer than 5 customers at the bar and no one waiting, an entering customer just increments `eating` and releases the mutex. The fifth customer sets `must_wait`.

If `must_wait` is set, entering customers block until the last customer at the bar clears `must_wait` and signals `block`. It is understood that the signaling thread gives up the mutex and the waiting thread receives it. Keep in mind, though, that this is an invariant understood by the programmer, and documented in the comments, but not enforced by the semantics of semaphores. It is up to us to get it right.

When the waiting thread resumes, we understand that it has the mutex. If there are other threads waiting, it signals `block` which, again, passes the mutex to a waiting thread. This process continues, with each thread passing the mutex to the next until there are no more chairs or no more waiting threads. In either case, the last thread releases the mutex and goes to sit down.

Reek calls this pattern "Pass the baton," since the mutex is being passed from one thread to the next like a baton in a relay race. One nice thing about this solution is that it is easy to confirm that updates to `eating` and `waiting` are consistent. A drawback is that it is harder to confirm that the mutex is being used correctly.

## 7.2 The child care problem

Max Hailperin wrote this problem for his textbook *Operating Systems and Middleware* [5]. At a child care center, state regulations require that there is always one adult present for every three children.

Puzzle: Write code for child threads and adult threads that enforces this constraint in a critical section.

### 7.2.1 Child care hint

Hailperin suggests that you can *almost* solve this problem with one semaphore.

Child care hint

```
1 multiplex = Semaphore(0)
```

`multiplex` counts the number of tokens available, where each token allows a child thread to enter. As adults enter, they signal `multiplex` three times; as they leave, they `wait` three times. But there is a problem with this solution.

Puzzle: what is the problem?

### 7.2.2 Child care non-solution

Here is what the adult code looks like in Hailperin's non-solution:

Child care non-solution (adult)

```
multiplex.signal(3)

# critical section

multiplex.wait()
multiplex.wait()
multiplex.wait()
```

The problem is a potential deadlock. Imagine that there are three children and two adults in the child care center. The value of `multiplex` is 3, so either adult should be able to leave. But if both adults start to leave at the same time, they might divide the available tokens between them, and both block.

Puzzle: solve this problem with a minimal change.

### 7.2.3 Child care solution

Adding a mutex solves the problem:

Child care solution (adult)

```
multiplex.signal(3)

# critical section

mutex.wait()
    multiplex.wait()
    multiplex.wait()
    multiplex.wait()
mutex.signal()
```

Now the three `wait` operations are atomic. If there are three tokens available, the thread that gets the mutex will get all three tokens and exit. If there are fewer tokens available, the first thread will block in the mutex and subsequent threads will queue on the mutex.

### 7.2.4 Extended child care problem

One feature of this solution is that an adult thread waiting to leave can prevent child threads from entering.

Imagine that there are 4 children and two adults, so the value of the multiplex is 2. If one of the adults tries to leave, she will take two tokens and then block waiting for the third. If a child thread arrives, it will wait even though it would be legal to enter. From the point of view of the adult trying to leave, that might be just fine, but if you are trying to maximize the utilization of the child care center, it's not.

Puzzle: write a solution to this problem that avoids unnecessary waiting.

Hint: think about the dancers in Section 3.8.

### 7.2.5 Extended child care hint

Here are the variables I used in my solution:

Extended child care hint

```
children = adults = waiting = leaving = 0
mutex = Semaphore(1)
childQueue = Semaphore(0)
adultQueue = Semaphore(0)
```

`children`, `adults`, `waiting` and `leaving` keep track of the number of children, adults, children waiting to enter, and adults waiting to leave; they are protected by `mutex`.

Children wait on `childQueue` to enter, if necessary. Adults wait on `adultQueue` to leave.

### 7.2.6 Extended child care solution

This solution is more complicated than Hailperin's elegant solution, but it is mostly a combination of patterns we have seen before: a scoreboard, two queues, and "I'll do it for you".

Here is the child code:

Extended child care solution (child)

```
mutex.wait()
    if children < 3 * adults:
        children++
        mutex.signal()
    else:
        waiting++
        mutex.signal()
        childQueue.wait()

# critical section

mutex.wait()
    children--
    if leaving and children <= 3 * (adults-1):
        leaving--
        adults--
        adultQueue.signal()
mutex.signal()
```

As children enter, they check whether there are enough adults and either (1) increment `children` and enter or (2) increment `waiting` and block. When they exit, they check for an adult thread waiting to leave and signal it if possible.

Here is the code for adults:

Extended child care solution (adult)

```
mutex.wait()
    adults++
    if waiting:
        n = min(3, waiting)
        childQueue.signal(n)
        waiting -= n
        children += n
mutex.signal()

# critical section

mutex.wait()
    if children <= 3 * (adults-1):
        adults--
        mutex.signal()
    else:
        leaving++
        mutex.signal()
        adultQueue.wait()
```

As adults enter, they signal waiting children, if any. Before they leave, they check whether there are enough adults left. If so, they decrement `adults` and exit. Otherwise they increment `leaving` and block. While an adult thread is waiting to leave, it counts as one of the adults in the critical section, so additional children can enter.

## 7.3 The room party problem

I wrote this problem while I was at Colby College. One semester there was a controversy over an allegation by a student that someone from the Dean of Students Office had searched his room in his absence. Although the allegation was public, the Dean of Students wasn't able to comment on the case, so we never found out what really happened. I wrote this problem to tease a friend of mine, who was the Dean of Student Housing.

The following synchronization constraints apply to students and the Dean of Students:

1. Any number of students can be in a room at the same time.
2. The Dean of Students can only enter a room if there are no students in the room (to conduct a search) or if there are more than 50 students in the room (to break up the party).
3. While the Dean of Students is in the room, no additional students may enter, but students may leave.
4. The Dean of Students may not leave the room until all students have left.
5. There is only one Dean of Students, so you do not have to enforce exclusion among multiple deans.

Puzzle: write synchronization code for students and for the Dean of Students that enforces all of these constraints.

### 7.3.1 Room party hint

Room party hint

```
students = 0
dean = 'not here'
mutex = Semaphore(1)
turn = Semaphore(1)
clear = Semaphore(0)
lieIn = Semaphore(0)
```

`students` counts the number of students in the room, and `dean` is the state of the Dean, which can also be "waiting" or "in the room". `mutex` protects `students` and `dean`, so this is yet another example of a scoreboard.

`turn` is a turnstile that keeps students from entering while the Dean is in the room.

`clear` and `lieIn` are used as rendezvouses between a student and the Dean (which is a whole other kind of scandal!).

### 7.3.2 Room party solution

This problem is hard. I worked through a lot of versions before I got to this one. The version that appeared in the first edition was mostly correct, but occasionally the Dean would enter the room and then find that he could neither search nor break up the party, so he would have to skulk off in embarrassed silence.

Matt Tesch wrote a solution that spared this humiliation, but the result was complicated enough that we had a hard time convincing ourselves that it was correct. But that solution led me to this one, which is a bit more readable.

Room party solution (dean)

```
mutex.wait()
    if students > 0 and students < 50:
        dean = 'waiting'
        mutex.signal()
        lieIn.wait()     # and get mutex from the student.

    # students must be 0 or >= 50

    if students >= 50:
        dean = 'in the room'
        breakup()
        turn.wait()      # lock the turnstile
        mutex.signal()
        clear.wait()     # and get mutex from the student.
        turn.signal()    # unlock the turnstile

    else:                # students must be 0
        search()

dean = 'not here'
mutex.signal()
```

When the Dean arrives, there are three cases: if there are students in the room, but not 50 or more, the Dean has to wait. If there are 50 or more, the Dean breaks up the party and waits for the students to leave. If there are no students, the Dean searches and leaves.

In the first two cases, the Dean has to wait for a rendezvous with a student, so he has to give up `mutex` to avoid a deadlock. When the Dean wakes up, he has to modify the scoreboard, so he needs to get the mutex back. This is similar to the situation we saw in the Sushi Bar problem. The solution I chose is the "Pass the baton" pattern.

Room party solution (student)

```
mutex.wait()
    if dean == 'in the room':
        mutex.signal()
        turn.wait()
        turn.signal()
        mutex.wait()

    students += 1

    if students == 50 and dean == 'waiting':
        lieIn.signal()
# and pass mutex to the dean
    else:
        mutex.signal()

party()

mutex.wait()
    students -= 1

    if students == 0 and dean == 'waiting':
        lieIn.signal()          # and pass mutex to the dean
    elif students == 0 and dean == 'in the room':
        clear.signal()          # and pass mutex to the dean
    else:
        mutex.signal()
```

There are three cases where a student might have to signal the Dean. If the Dean is waiting, then the 50th student in or the last one out has to signal `lieIn`. If the Dean is in the room (waiting for all the students to leave), the last student out signals `clear`. In all three cases, it is understood that the mutex passes from the student to the Dean.

One part of this solution that may not be obvious is how we know at Line 7 of the Dean's code that `students` must be 0 or not less than 50. The key is to realize that there are only two ways to get to this point: either the first conditional was false, which means that `students` is either 0 or not less than 50; or the Dean was waiting on `lieIn` when a student signaled, which means, again, that `students` is either 0 or not less than 50.

## 7.4 The Senate Bus problem

This problem was originally based on the Senate bus at Wellesley College. Riders come to a bus stop and wait for a bus. When the bus arrives, all the waiting riders invoke `boardBus`, but anyone who arrives while the bus is boarding has to wait for the next bus. The capacity of the bus is 50 people; if there are more than 50 people waiting, some will have to wait for the next bus.

When all the waiting riders have boarded, the bus can invoke `depart`. If the bus arrives when there are no riders, it should depart immediately.

Puzzle: Write synchronization code that enforces all of these constraints.

### 7.4.1 Bus problem hint

Here are the variables I used in my solution:

Bus problem hint

```
riders = 0
mutex = Semaphore(1)
multiplex = Semaphore(50)
bus = Semaphore(0)
allAboard = Semaphore(0)
```

`mutex` protects `riders`, which keeps track of how many riders are waiting; `multiplex` makes sure there are no more than 50 riders in the boarding area.

Riders wait on `bus`, which gets signaled when the bus arrives. The bus waits on `allAboard`, which gets signaled by the last student to board.

### 7.4.2 Bus problem solution #1

Here is the code for the bus. Again, we are using the "Pass the baton" pattern.

Bus problem solution (bus)

```
mutex.wait()
if riders > 0:
    bus.signal()         # and pass the mutex
    allAboard.wait()     # and get the mutex back
mutex.signal()

depart()
```

When the bus arrives, it gets `mutex`, which prevents late arrivals from entering the boarding area. If there are no riders, it departs immediately. Otherwise, it signals `bus` and waits for the riders to board.

Here is the code for the riders:

Bus problem solution (riders)

```
multiplex.wait()
    mutex.wait()
        riders += 1
    mutex.signal()

    bus.wait()               # and get the mutex
multiplex.signal()

boardBus()

riders -= 1
if riders == 0:
    allAboard.signal()
else:
    bus.signal()             # and pass the mutex
```

The multiplex controls the number of riders in the waiting area, although strictly speaking, a rider doesn't enter the waiting area until she increments `riders`.

Riders wait on `bus` until the bus arrives. When a rider wakes up, it is understood that she has the mutex. After boarding, each rider decrements `riders`. If there are more riders waiting, the boarding rider signals `bus` and pass the mutex to the next rider. The last rider signals `allAboard` and passes the mutex back to the bus.

Finally, the bus releases the mutex and departs.

Puzzle: can you find a solution to this problem using the "I'll do it for you" pattern?

### 7.4.3 Bus problem solution #2

Grant Hutchins came up with this solution, which uses fewer variables than the previous one, and doesn't involve passing around any mutexes. Here are the variables:

Bus problem solution #2 (initialization)

```
waiting = 0
mutex = Semaphore(1)
bus = Semaphore(0)
boarded = Semaphore(0)
```

`waiting` is the number of riders in the boarding area, which is protected by `mutex`. `bus` signals when the bus has arrived; `boarded` signals that a rider has boarded.

Here is the code for the bus.

Bus problem solution (bus)

```
mutex.wait()
n = min(waiting, 50)
for i in range(n):
    bus.signal()
    boarded.wait()

waiting = max(waiting-50, 0)
mutex.signal()

depart()
```

The bus gets the `mutex` and holds it throughout the boarding process. The loop signals each rider in turn and waits for her to board. By controlling the number of signals, the bus prevents more than 50 riders from boarding.

When all the riders have boarded, the bus updates `waiting`, which is an example of the "I'll do it for you" pattern.

The code for the riders uses two simple patterns: a mutex and a rendezvous.

Bus problem solution (riders)

```
mutex.wait()
     waiting += 1
mutex.signal()

bus.wait()
board()
boarded.signal()
```

Challenge: if riders arrive while the bus is boarding, they might be annoyed if you make them wait for the next one. Can you find a solution that allows late arrivals to board without violating the other constraints?

## 7.5 The Faneuil Hall problem

This problem was written by Grant Hutchins, who was inspired by a friend who took her Oath of Citizenship at Faneuil Hall in Boston.

"There are three kinds of threads: immigrants, spectators, and a one judge. Immigrants must wait in line, check in, and then sit down. At some point, the judge enters the building. When the judge is in the building, no one may enter, and the immigrants may not leave. Spectators may leave. Once all immigrants check in, the judge can confirm the naturalization. After the confirmation, the immigrants pick up their certificates of U.S. Citizenship. The judge leaves at some point after the confirmation. Spectators may now enter as before. After immigrants get their certificates, they may leave."

To make these requirements more specific, let's give the threads some functions to execute, and put constraints on those functions.

- Immigrants must invoke `enter`, `checkIn`, `sitDown`, `swear`, `getCertificate` and `leave`.
- The judge invokes `enter`, `confirm` and `leave`.
- Spectators invoke `enter`, `spectate` and `leave`.
- While the judge is in the building, no one may `enter` and immigrants may not `leave`.
- The judge can not `confirm` until all immigrants who have invoked `enter` have also invoked `checkIn`.
- Immigrants can not `getCertificate` until the judge has executed `confirm`.

### 7.5.1 Faneuil Hall Problem Hint

Faneuil Hall problem hint

```
noJudge = Semaphore(1)
entered = 0
checked = 0
mutex = Semaphore(1)
confirmed = Semaphore(0)
```

`noJudge` acts as a turnstile for incoming immigrants and spectators; it also protects `entered`, which counts the number of immigrants in the room. `checked` counts the number of immigrants who have checked in; it is protected by `mutex`.

`confirmed` signals that the judge has executed `confirm`.

### 7.5.2 Faneuil Hall problem solution

Here is the code for immigrants:

Faneuil Hall problem solution (immigrant)

```
noJudge.wait()
enter()
entered++
noJudge.signal()

mutex.wait()
checkIn()
checked++

if judge == 1 and entered == checked:
    allSignedIn.signal()
# and pass the mutex
else:
    mutex.signal()

sitDown()
confirmed.wait()

swear()
getCertificate()

noJudge.wait()
leave()
noJudge.signal()
```

Immigrants pass through a turnstile when they enter; while the judge is in the room, the turnstile is locked.

After entering, immigrants have to get `mutex` to check in and update `checked`. If there is a judge waiting, the last immigrant to check in signals `allSignedIn` and passes the mutex to the judge.

Here is the code for the judge:

Faneuil Hall problem solution (judge)

```
noJudge.wait()
mutex.wait()

enter()
judge = 1

if entered > checked:
    mutex.signal()
```

```
    allSignedIn.wait()
# and get the mutex back.

confirm()
confirmed.signal(checked)
entered = checked = 0

leave()
judge = 0

mutex.signal()
noJudge.signal()
```

The judge holds `noJudge` to bar immigrants and spectators from entering, and `mutex` so he can access `entered` and `checked`.

If the judge arrives at an instant when everyone who has entered has also checked in, she can proceed immediately. Otherwise, she has to give up the mutex and wait. When the last immigrant checks in and signals `allSignedIn`, it is understood that the judge will get the mutex back.

After invoking `confirm`, the judge signals `confirmed` once for every immigrant who has checked in, and then resets the counters (an example of "I'll do it for you"). Then the judge leaves and releases `mutex` and `noJudge`.

After the judge signals `confirmed`, immigrants invoke `swear` and `getCertificate` concurrently, and then wait for the `noJudge` turnstile to open before leaving.

The code for spectators is easy; the only constraint they have to obey is the `noJudge` turnstile.

Faneuil Hall problem solution (spectator)

```
noJudge.wait()
enter()
noJudge.signal()

spectate()

leave()
```

Note: in this solution it is possible for immigrants to get stuck, after they get their certificate, by another judge coming to swear in the next batch of immigrants. If that happens, they might have to wait through another swearing in-ceremony.

Puzzle: modify this solution to handle the additional constraint that after the judge leaves, all immigrants who have been sworn in must leave before the judge can enter again.

### 7.5.3 Extended Faneuil Hall Problem Hint

My solution uses the following additional variables:

Faneuil Hall problem hint

```
exit = Semaphore(0)
allGone = Semaphore(0)
```

Since the extended problem involves an additional rendezvous, we can solve it with two semaphores.

One other hint: I found it useful to use the "pass the baton" pattern again.

### 7.5.4 Extended Faneuil Hall problem solution

The top half of this solution is the same as before. The difference starts at Line 21. Immigrants wait here for the judge to leave.

Faneuil Hall problem solution (immigrant)

```
1  noJudge.wait()
2  enter()
3  entered++
4  noJudge.signal()
5
6  mutex.wait()
7  checkIn()
8  checked++
9
10 if judge = 1 and entered == checked:
11     allSignedIn.signal()
   # and pass the mutex
12 else:
13     mutex.signal()
14
15 sitDown()
16 confirmed.wait()
17
18 swear()
19 getCertificate()
20
21 exit.wait()
   # and get the mutex
22 leave()
23 checked--
24 if checked == 0:
25     allGone.signal()
   # and pass the mutex
26 else:
27     exit.signal()
   # and pass the mutex
```

For the judge, the difference starts at Line 18. When the judge is ready to leave, she can't release `noJudge`, because that would allow more immigrants, and possibly another judge, to enter. Instead, she signals `exit`, which allows one immigrant to leave, and passes `mutex`.

The immigrant that gets the signal decrements `checked` and then passes the baton to the next immigrant. The last immigrant to leave signals `allGone` and passes the mutex back to the judge. This pass-back is not strictly necessary, but it has the nice feature that the judge releases both `mutex` and `noJudge` to end the phase cleanly.

Faneuil Hall problem solution (judge)

```
noJudge.wait()
mutex.wait()

enter()
judge = 1

if entered > checked:
    mutex.signal()
    allSignedIn.wait()
# and get the mutex back.

confirm()
confirmed.signal(checked)
entered = 0

leave()
judge = 0

exit.signal()
# and pass the mutex
allGone.wait()                        # and get it back
mutex.signal()
noJudge.signal()
```

The spectator code for the extended problem is unchanged.

## 7.6 Dining Hall problem

This problem was written by Jon Pollack during my Synchronization class at Olin College.

Students in the dining hall invoke `dine` and then `leave`. After invoking `dine` and before invoking `leave` a student is considered "ready to leave".

The synchronization constraint that applies to students is that, in order to maintain the illusion of social suave, a student may never sit at a table alone. A student is considered to be sitting alone if everyone else who has invoked `dine` invokes `leave` before she has finished `dine`.

Puzzle: write code that enforces this constraint.

### 7.6.1 Dining Hall problem hint

Dining Hall problem hint

```
eating = 0
readyToLeave = 0
mutex = Semaphore(1)
okToLeave = Semaphore(0)
```

`eating` and `readyToLeave` are counters protected by `mutex`, so this is the usual scoreboard pattern.

If a student is ready to leave, but another student would be left alone at the table, she waits on `okToLeave` until another student changes the situation and signals.

### 7.6.2 Dining Hall problem solution

If you analyze the constraints, you will realize that there is only one situation where a student has to wait, if there is one student eating and one student who wants to leave. But there are two ways to get out of this situation: another student might arrive to eat, or the dining student might finish.

In either case, the student who signals the waiting student updates the counters, so the waiting student doesn't have to get the mutex back. This is another example of the the "I'll do it for you" pattern.

Dining Hall problem solution

```
getFood()

mutex.wait()
eating++
if eating == 2 and readyToLeave == 1:
    okToLeave.signal()
    readyToLeave--
mutex.signal()

dine()

mutex.wait()
eating--
readyToLeave++

if eating == 1 and readyToLeave == 1:
    mutex.signal()
    okToLeave.wait()
elif eating == 0 and readyToLeave == 2:
    okToLeave.signal()
    readyToLeave -= 2
    mutex.signal()
else:
    readyToLeave--
    mutex.signal()

leave()
```

When a student is checking in, if she sees one student eating and one waiting to leave, she lets the waiter off the hook and decrements `readyToLeave` for him.

After dining, the student checks three cases:

- If there is only one student left eating, the departing student has to give up the mutex and wait.
- If the departing student finds that someone is waiting for her, she signals him and updates the counter for both of them.
- Otherwise, she just decrements `readyToLeave` and leaves.

## 7.7 Extended Dining Hall problem

The Dining Hall problem gets a little more challenging if we add another step. As students come to lunch they invoke `getFood`, `dine` and then `leave`. After invoking `getFood` and before invoking `dine`, a student is considered "ready to eat". Similarly, after invoking `dine` a student is considered "ready to leave".

The same synchronization constraint applies: a student may never sit at a table alone. A student is considered to be sitting alone if either

- She invokes `dine` while there is no one else at the table and no one ready to eat, or
- everyone else who has invoked `dine` invokes `leave` before she has finished `dine`.

Puzzle: write code that enforces these constraints.

### 7.7.1 Extended Dining Hall problem hint

Here are the variables I used in my solution:

Extended Dining Hall problem hint

```
readyToEat = 0
eating = 0
readyToLeave = 0
mutex = Semaphore(1)
okToSit = Semaphore(0)
okToLeave = Semaphore(0)
```

`readyToEat`, `eating` and `readyToLeave` are counters, all protected by `mutex`.

If a student is in a situation where she cannot proceed, she waits on `okToSit` or `okToLeave` until another student changes the situation and signals.

I also used a per-thread variable named `hasMutex` to help keep track of whether or not a thread holds the mutex.

### 7.7.2 Extended Dining Hall problem solution

Again, if we analyze the constraints, we realize that there is only one situation where a student who is ready to eat has to wait, if there is no one eating and no one else ready to eat. And the only way out is if someone else arrives who is ready to eat.

Extended Dining Hall problem solution

```
getFood()

mutex.wait()
readyToEat++
if eating == 0 and readyToEat == 1:
    mutex.signal()
    okToSit.wait()
elif eating == 0 and readyToEat == 2:
    okToSit.signal()
    readyToEat -= 2
    eating += 2
    mutex.signal()
else:
    readyToEat--
    eating++
    if eating == 2 and readyToLeave == 1:
        okToLeave.signal()
        readyToLeave--
    mutex.signal()

dine()

mutex.wait()
eating--
readyToLeave++
if eating == 1 and readyToLeave == 1:
    mutex.signal()
    okToLeave.wait()
elif eating == 0 and readyToLeave == 2:
    okToLeave.signal()
    readyToLeave -= 2
    mutex.signal()
else:
    readyToLeave--
    mutex.signal()

leave()
```

As in the previous solution, I used the "I'll do it for you" pattern so that a waiting student doesn't have to get the mutex back.

The primary difference between this solution and the previous one is that the first student who arrives at an empty table has to wait, and the second student allows both students to proceed. In either case, we don't have to check for students waiting to leave, since no one can leave an empty table!

# Chapter 8

# Synchronization in Python

By using pseudocode, we have avoided some of the ugly details of synchronization in the real world. In this chapter we'll look at real synchronization code in Python; in the next chapter we'll look at C.

Python provides a reasonably pleasant multithreading environment, complete with Semaphore objects. It has a few foibles, but there is some cleanup code in Appendix A that makes things a little better.

Here is a simple example:

Listing:

```
from threading_cleanup import *

class Shared:
    def __init__(self):
        self.counter = 0

def child_code(shared):
    while True:
        shared.counter += 1
        print shared.counter
        time.sleep(0.5)

shared = Shared()
children = [Thread(child_code, shared) for i in range(2)]
for child in children: child.join()
```

The first line runs the cleanup code from Appendix A; I will leave this line out of the other examples.

`Shared` defines an object type that will contain shared variables. Global variables are also shared between threads, but we won't use any in these examples. Variables that are local in the sense that they are declared inside a function are also local in the sense that they are thread-specific.

The child code is an infinite loop that increments `counter`, prints the new value, and then sleeps for 0.5 seconds.

The parent thread creates `shared` and two children, then waits for the children to exit (which in this case, they won't).

## 8.1 Mutex checker problem

Diligent students of synchronization will notice that the children make unsynchronized updates to `counter`, which is not safe! If you run this program, you might see some errors, but you probably won't. The nasty thing about synchronization errors is that they are unpredictable, which means that even extensive testing may not reveal them.

To detect errors, it is often necessary to automate the search. In this case, we can detect errors by keeping track of the values of `counter`.

Listing:

```
class Shared:
    def __init__(self, end=10):
        self.counter = 0
        self.end = end
        self.array = [0]* self.end

def child_code(shared):
    while True:
        if shared.counter >= shared.end: break
        shared.array[shared.counter] += 1
        shared.counter += 1

shared = Shared(10)
children = [Thread(child_code, shared) for i in range(2)]
for child in children: child.join()
print shared.array
```

In this example, `shared` contains a list (misleadingly named `array`) that keeps track of the number of times each value of counter is used. Each time through the loop, the children check `counter` and quit if it exceeds `end`. If not, they use `counter` as an index into `array` and increment the corresponding entry. Then they increment `counter`.

If everything works correctly, each entry in the array should be incremented exactly once. When the children exit, the parent returns from `join` and prints the value of `array`. When I ran the program, I got

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

which is disappointingly correct. If we increase the size of the array, we might expect more errors, but it also gets harder to check the result.

We can automate the checker by making a histogram of the results in the array:

Listing:

```
class Histogram(dict):
    def __init__(self, seq=[]):
        for item in seq:
            self[item] = self.get(item, 0) + 1

print Histogram(shared.array)
```

Now when I run the program, I get

```
{1: 10}
```

which means that the value 1 appeared 10 times, as expected. No errors so far, but if we make `end` bigger, things get more interesting:

```
end = 100, {1: 100}
end = 1000, {1: 1000}
end = 10000, {1: 10000}
end = 100000, {1: 27561, 2: 72439}
```

Oops! When `end` is big enough that there are a lot of context switches between the children, we start to get synchronization errors. In this case, we get a *lot* of errors, which suggests that the program falls into a recurring pattern where threads are consistently interrupted in the critical section.

This example demonstrates one of the dangers of synchronization errors, which is that they may be rare, but they are not random. If an error occurs one time in a million, that doesn't mean it won't happen a million times in a row.

Puzzle: add synchronization code to this program to enforce exclusive access to the shared variables. You can download the code in this section from greenteapress.com/sem2/2

### 8.1.1 Mutex checker hint

Here is the version of `Shared` I used:

Listing:

```
class Shared:
    def __init__(self, end=10):
        self.counter = 0
        self.end = end
        self.array = [0]* self.end
        self.mutex = Semaphore(1)
```

The only change is the Semaphore named `mutex`, which should come as no surprise.

### 8.1.2 Mutex checker solution

Here is my solution:

Listing:

```
def child_code(shared):
    while True:
        shared.mutex.wait()
        if shared.counter < shared.end:
            shared.array[shared.counter] += 1
            shared.counter += 1
            shared.mutex.signal()
        else:
            shared.mutex.signal()
            break
```

Although this is not the most difficult synchronization problem in this book, you might have found it tricky to get the details right. In particular, it is easy to forget to signal the mutex before breaking out of the loop, which would cause a deadlock.

I ran this solution with `end = 1000000`, and got the following result:

```
{1: 1000000}
```

Of course, that doesn't mean my solution is correct, but it is off to a good start.

## 8.2 The coke machine problem

The following program simulates producers and consumers adding and removing cokes from a coke machine:

Listing:

```
import random

class Shared:
    def __init__(self, start=5):
        self.cokes = start

def consume(shared):
    shared.cokes -= 1
    print shared.cokes

def produce(shared):
    shared.cokes += 1
    print shared.cokes

def loop(shared, f, mu=1):
    while True:
        t = random.expovariate(1.0/mu)
        time.sleep(t)
        f(shared)

shared = Shared()
fs = [consume]*2 + [produce]*2
threads = [Thread(loop, shared, f) for f in fs]
for thread in threads: thread.join()
```

The capacity is 10 cokes, and that the machine is initially half full. So the shared variable `cokes` is 5.

The program creates 4 threads, two producers and two consumers. They both run `loop`, but producers invoke `produce` and consumers invoke `consume`. These functions make unsynchronized access to a shared variable, which is a no-no.

Each time through the loop, producers and consumers sleep for a duration chosen from an exponential distribution with mean `mu`. Since there are two producers and two consumers, two cokes get added to the machine per second, on average, and two get removed.

So on average the number of cokes is constant, but in the short run in can vary quite widely. If you run the program for a while, you will probably see the value of `cokes` dip below zero, or climb above 10. Of course, neither of these should happen.

Puzzle: add code to this program to enforce the following synchronization constraints:

- Access to `cokes` should be mutually exclusive.
- If the number of cokes is zero, consumers should block until a coke is added.
- If the number of cokes is 10, producers should block until a coke is removed.

You can download the program from greenteapress.com/sem2/3

### 8.2.1 Coke machine hint

Here are the shared variables I used in my solution:

Listing:

```
class Shared:
    def __init__(self, start=5, capacity=10):
        self.cokes = Semaphore(start)
        self.slots = Semaphore(capacity-start)
        self.mutex = Semaphore(1)
```

`cokes` is a Semaphore now (rather than a simple integer), which makes it tricky to print its value. Of course, you should never access the value of a Semaphore, and Python in its usual do-gooder way doesn't provide any of the cheater methods you see in some implementations.

But you might find it interesting to know that the value of a Semaphore is stored in a private attribute named `_Semaphore__value`. Also, in case you don't know, Python doesn't actually enforce any restriction on access to private attributes. You should never access it, of course, but I thought you might be interested.

Ahem.

### 8.2.2 Coke machine solution

If you've read the rest of this book, you should have had no trouble coming up with something at least as good as this:

Listing:

```
def consume(shared):
    shared.cokes.wait()
    shared.mutex.wait()
    print shared.cokes.value()
    shared.mutex.signal()
    shared.slots.signal()

def produce(shared):
    shared.slots.wait()
    shared.mutex.wait()
    print shared.cokes._Semaphore__value
    shared.mutex.signal()
    shared.cokes.signal()
```

If you run this program for a while, you should be able to confirm that the number of cokes in the machine is never negative or greater than 10. So this solution seems to be correct.

So far.

# Chapter 9

# Synchronization in C

In this section we will write a multithreaded, synchronized program in C. Appendix B provides some of the utility code I use to make the C code a little more palatable. The examples in this section depend on that code.

## 9.1 Mutual exclusion

We'll start by defining a structure that contains shared variables:

Listing:

```
typedef struct {
  int counter;
  int end;
  int *array;
} Shared;

Shared *make_shared (int end)
{
  int i;
  Shared *shared = check_malloc (sizeof (Shared));

  shared->counter = 0;
  shared->end = end;

  shared->array = check_malloc (shared->end * sizeof(int));
  for (i=0; i<shared->end; i++) {
    shared->array[i] = 0;
  }
  return shared;
}
```

`counter` is a shared variable that will be incremented by concurrent threads until it reaches `end`. We will use `array` to check for synchronization errors by keeping track of the value of `counter` after each increment.

### 9.1.1 Parent code

Here is the code the parent thread uses to create threads and wait for them to complete:

Listing:

```
int main ()
{
  int i;
  pthread_t child[NUM_CHILDREN];

  Shared *shared = make_shared (100000);

  for (i=0; i<NUM_CHILDREN; i++) {
    child[i] = make_thread (entry, shared);
  }

  for (i=0; i<NUM_CHILDREN; i++) {
    join_thread (child[i]);
  }

  check_array (shared);
  return 0;
}
```

The first loop creates the child threads; the second loop waits for them to complete. When the last child has finished, the parent invokes `check_array` to check for errors. `make_thread` and `join_thread` are defined in Appendix B.

### 9.1.2 Child code

Here is the code that is executed by each of the children:

Listing:

```
void child_code (Shared *shared)
{
  while (1) {
    if (shared->counter >= shared->end) {
      return;
    }
    shared->array[shared->counter]++;
    shared->counter++;
```

```
  }
}
```

Each time through the loop, the child threads use `counter` as an index into `array` and increment the corresponding element. Then they increment `counter` and check to see if they're done.

### 9.1.3 Synchronization errors

If everything works correctly, each element of the array should be incremented once. So to check for errors, we can just count the number of elements that are not 1:

Listing:

```
void check_array (Shared *shared)
{
  int i, errors=0;

  for (i=0; i<shared->end; i++) {
    if (shared->array[i] != 1) errors++;
  }
  printf ("%d errors.\n", errors);
}
```

You can download this program (including the cleanup code) from greenteapress.com/sem2/4

If you compile and run the program, you should see output like this:

```
Starting child at counter 0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Child done.
Starting child at counter 100000
Child done.
Checking...
0 errors.
```

Of course, the interaction of the children depends on details of your operating system and also other programs running on your computer. In the example

shown here, one thread ran all the way from 0 to `end` before the other thread got started, so it is not surprising that there were no synchronization errors.

But as `end` gets bigger, there are more context switches between the children. On my system I start to see errors when `end` is 100,000,000.

Puzzle: use semaphores to enforce exclusive access to the shared variables and run the program again to confirm that there are no errors.

### 9.1.4 Mutual exclusion hint

Here is the version of `Shared` I used in my solution:

Listing:

```
1  typedef struct {
2    int counter;
3    int end;
4    int *array;
5    Semaphore *mutex;
6  } Shared;
7
8  Shared *make_shared (int end)
9  {
10   int i;
11   Shared *shared = check_malloc (sizeof (Shared));
12
13   shared->counter = 0;
14   shared->end = end;
15
16   shared->array = check_malloc (shared->end * sizeof(int));
17   for (i=0; i<shared->end; i++) {
18     shared->array[i] = 0;
19   }
20   shared->mutex = make_semaphore(1);
21   return shared;
22 }
```

Line 5 declares `mutex` as a Semaphore; Line 20 initializes the mutex with the value 1.

### 9.1.5 Mutual exclusion solution

Here is the synchronized version of the child code:

Listing:

```
void child_code (Shared *shared)
{
  while (1) {
    sem_wait(shared->mutex);
    if (shared->counter >= shared->end) {
      sem_signal(shared->mutex);
      return;
    }

    shared->array[shared->counter]++;
    shared->counter++;
    sem_signal(shared->mutex);
  }
}
```

There is nothing too surprising here; the only tricky thing is to remember to release the mutex before the `return` statement.

You can download this solution from greenteapress.com/sem2/5

## 9.2 Make your own semaphores

The most commonly used synchronization tools for programs that use Pthreads are mutexes and condition variables, not semaphores. For an explanation of these tools, I recommend Butenhof's *Programming with POSIX Threads* [2].

Puzzle: read about mutexes and condition variables, and then use them to write an implementation of semaphores.

You might want to use the following utility code in your solutions. Here is my wrapper for Pthreads mutexes:

Listing:

```
typedef pthread_mutex_t Mutex;

Mutex *make_mutex ()
{
  Mutex *mutex = check_malloc (sizeof(Mutex));
  int n = pthread_mutex_init (mutex, NULL);
  if (n != 0) perror_exit ("make_lock failed");
  return mutex;
}

void mutex_lock (Mutex *mutex)
{
  int n = pthread_mutex_lock (mutex);
  if (n != 0) perror_exit ("lock failed");
}

void mutex_unlock (Mutex *mutex)
{
  int n = pthread_mutex_unlock (mutex);
  if (n != 0) perror_exit ("unlock failed");
}
```

And my wrapper for Pthread condition variables:

Listing:

```
typedef pthread_cond_t Cond;

Cond *make_cond ()
{
  Cond *cond = check_malloc (sizeof(Cond));
  int n = pthread_cond_init (cond, NULL);
  if (n != 0) perror_exit ("make_cond failed");
  return cond;
}

void cond_wait (Cond *cond, Mutex *mutex)
{
  int n = pthread_cond_wait (cond, mutex);
  if (n != 0) perror_exit ("cond_wait failed");
}

void cond_signal (Cond *cond)
{
  int n = pthread_cond_signal (cond);
  if (n != 0) perror_exit ("cond_signal failed");
}
```

### 9.2.1 Semaphore implementation hint

Here is the structure definition I used for my semaphores:

Listing:

```
typedef struct {
  int value, wakeups;
  Mutex *mutex;
  Cond *cond;
} Semaphore;
```

`value` is the value of the semaphore. `wakeups` counts the number of pending signals; that is, the number of threads that have been woken but have not yet resumed execution. The reason for wakeups is to make sure that our semaphores have Property 3, described in Section 4.4.

`mutex` provides exclusive access to `value` and `wakeups`; `cond` is the condition variable threads wait on if they wait on the semaphore.

Here is the initialization code for this structure:

Listing:

```
Semaphore *make_semaphore (int value)
{
  Semaphore *semaphore = check_malloc (sizeof(Semaphore));
  semaphore->value = value;
  semaphore->wakeups = 0;
  semaphore->mutex = make_mutex ();
  semaphore->cond = make_cond ();
  return semaphore;
}
```

### 9.2.2 Semaphore implementation

Here is my implementation of semaphores using Pthread's mutexes and condition variables:

Listing:

```
1  void sem_wait (Semaphore *semaphore)
2  {
3    mutex_lock (semaphore->mutex);
4    semaphore->value--;
5
6    if (semaphore->value < 0) {
7      do {
8        cond_wait (semaphore->cond, semaphore->mutex);
9      } while (semaphore->wakeups < 1);
10     semaphore->wakeups--;
11   }
12   mutex_unlock (semaphore->mutex);
13 }
14
15 void sem_signal (Semaphore *semaphore)
16 {
17   mutex_lock (semaphore->mutex);
18   semaphore->value++;
19
20   if (semaphore->value <= 0) {
21     semaphore->wakeups++;
22     cond_signal (semaphore->cond);
23   }
24   mutex_unlock (semaphore->mutex);
25 }
```

Most of this is straightforward; the only thing that might be tricky is the `do...while` loop at Line 7. This is an unusual way to use a condition variable, but in this case it is necessary.

Puzzle: why can't we replace this `do...while` loop with a `while` loop?

### 9.2.3 Semaphore implementation detail

With a `while` loop, this implementation would not have Property 3. It would be possible for a thread to signal and then run around and catch its own signal.

With the `do...while` loop, it is guaranteed[1] that when a thread signals, one of the waiting threads will get the signal, even if another thread gets the mutex at Line 3 before one of the waiting threads resumes.

[1] Well, almost. It turns out that a well-timed spurious wakeup (see greenteapress.com/sem2/6) can violate this guarantee.

# Bibliography


[1] Gregory R. Andrews. *Concurrent Programming: Principles and Practice*. Addison-Wesley, 1991.

[2] David R. Butenhof. *Programming with POSIX Threads*. Addison-Wesley, 1997.

[3] Edsger Dijkstra. Cooperating sequential processes. 1965. Reprinted in *Programming Languages*, F. Genuys, ed., Academic Press, New York 1968.

[4] Armando R. Gingras. Dining philosophers revisited. *ACM SIGCSE Bulletin*, 22(3):21–24, 28, September 1990.

[5] Max Hailperin. *Operating Systems and Middleware: Supporting Controlled Interaction*. Thompson Course Technology, 2006.

[6] Joseph M. Morris. A starvation-free solution to the mutual exclusion problem. *Information Processing Letters*, 8:76–80, February 1979.

[7] David L. Parnas. On a solution to the cigarette smokers' problem without conditional statements. *Communications of the ACM*, 18:181–183, March 1975.

[8] Suhas Patil. Limitations and capabilities of Dijkstra's semaphore primitives for coordination among processes. Technical report, MIT, 1971.

[9] Kenneth A. Reek. Design patterns for semaphores. In *ACM SIGCSE*, 2004.

[10] Abraham Silberschatz and Peter Baer Galvin. *Operating Systems Concepts*. Addison Wesley, fifth edition, 1997.

[11] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, fourth edition, 2000.

[12] Andrew S. Tanenbaum. *Modern Operating Systems*. Prentice Hall, second edition, 2001.

# Appendix A

# Cleaning up Python threads

Compared to a lot of other threading environments, Python threads are pretty good, but there are a couple of features that annoy me. Fortunately, you can fix them with a little clean-up code.

## A.1 Semaphore methods

First, the methods for Python semaphores are called `acquire` and `release`, which is a perfectly reasonable choice, but after working on this book for a couple of years, I am used to `signal` and `wait`. Fortunately, I can have it my way by subclassing the version of Semaphore in the `threading` module:

Semaphore name change

```
import threading

class Semaphore(threading._Semaphore):
    wait = threading._Semaphore.acquire
    signal = threading._Semaphore.release
```

Once this class is defined, you can create and manipulate Semaphores using the syntax in this book.

Semaphore example

```
mutex = Semaphore()
mutex.wait()
mutex.signal()
```

## A.2 Creating threads

The other feature of the `threading` module that annoys me is the interface for creating and starting threads. The usual way requires keyword arguments and

two steps:

Thread example (standard way)

```
import threading

def function(x, y, z):
    print x, y, z

thread = threading.Thread(target=function, args=[1, 2, 3])
thread.start()
```

In this example, creating the thread has no immediate effect. But when you invoke `start`, the new thread executes the target function with the given arguments. This is great if you need to do something with the thread before it starts, but I almost never do. Also, I think the keyword arguments `target` and `args` are awkward.

Fortunately, we can solve both of these problems with four lines of code.

Cleaned-up Thread class

```
class Thread(threading.Thread):
    def __init__(self, t, *args):
        threading.Thread.__init__(self, target=t, args=args)
        self.start()
```

Now we can create threads with a nicer interface, and they start automatically:

Thread example (my way)

```
thread = Thread(function, 1, 2, 3)
```

This also lends itself to an idiom I like, which is to create multiple Threads with a list comprehension:

Multiple thread example

```
threads = [Thread(function, i, i, i) for i in range(10)]
```

## A.3 Handling keyboard interrupts

One other problem with the `threading` class is that `Thread.join` can't be interrupted by Ctrl-C, which generates the signal `SIGINT`, which Python translates into a KeyboardInterrupt.

So, if you write the following program:

Unstoppable program

```
import threading, time

class Thread(threading.Thread):
    def __init__(self, t, *args):
        threading.Thread.__init__(self, target=t, args=args)
        self.start()

def parent_code():
    child = Thread(child_code, 10)
    child.join()

def child_code(n=10):
    for i in range(n):
        print i
        time.sleep(1)

parent_code()
```

You will find that it cannot be interrupted with Ctrl-C or a `SIGINT`[1].

My workaround for this problem uses `os.fork` and `os.wait`, so it only works on UNIX and Macintosh. Here's how it works: before creating new threads, the program invokes `watcher`, which forks a new process. The new process returns and executes the rest of the program. The original process waits for the child process to complete, hence the name `watcher`:

The watcher

```
import threading, time, os, signal, sys

class Thread(threading.Thread):
    def __init__(self, t, *args):
        threading.Thread.__init__(self, target=t, args=args)
        self.start()

def parent_code():
    child = Thread(child_code, 10)
    child.join()

def child_code(n=10):
    for i in range(n):
        print i
        time.sleep(1)
```

[1] At the time of this writing, this bug had been reported and assigned number 1167930, but it was open and unassigned (greenteapress.com/sem2/7).

```
16
17 def watcher():
18     child = os.fork()
19     if child == 0: return
20     try:
21         os.wait()
22     except KeyboardInterrupt:
23         print 'KeyboardInterrupt'
24         os.kill(child, signal.SIGKILL)
25     sys.exit()
26
27 watcher()
28 parent_code()
```

If you run this version of the program, you should be able to interrupt it with Ctrl-C. I am not sure, but I think it is guaranteed that the `SIGINT` is delivered to the watcher process, so that's one less thing the parent and child threads have to deal with.

I keep all this code in a file named `threading_cleanup.py`, which you can download from greenteapress.com/sem2/8

The examples in Chapter 8 are presented with the understanding that this code executes prior to the example code.

# Appendix B

# Cleaning up POSIX threads

In this section, I present some utility code I use to make multithreading in C a little more pleasant. The examples in Section 9 are based on this code.

Probably the most popular threading standard used with C is POSIX Threads, or Pthreads for short. The POSIX standard defines a thread model and an interface for creating and controlling threads. Most versions of UNIX provide an implementation of Pthreads.

## B.1 Compiling Pthread code

Using Pthreads is like using most C libraries:

- You include headers files at the beginning of your program.
- You write code that calls functions defined by Pthreads.
- When you compile the program, you link it with the Pthread library.

For my examples, I include the following headers:

Headers

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
```

The first two are standard; the third is for Pthreads and the fourth is for semaphores. To compile with the Pthread library in `gcc`, you can use the `-l` option on the command line:

Listing:

```
gcc -g -O2 -o array array.c -lpthread
```

This compiles `array.c` with debugging info and optimization, links with the Pthread library, and generates an executable named `array`.

If you are used to a language like Python that provides exception handling, you will probably be annoyed with languages like C that require you to check for error conditions explicitly. I often mitigate this hassle by wrapping library function calls together with their error-checking code inside my own functions. For example, here is a version of `malloc` that checks the return value.

Listing:

```
void *check_malloc(int size)
{
  void *p = malloc (size);
  if (p == NULL) {
    perror ("malloc failed");
    exit (-1);
  }
  return p;
}
```

## B.2 Creating threads

I've done the same thing with the Pthread functions I'm going to use; here's my wrapper for `pthread_create`.

Listing:

```
pthread_t make_thread(void *(*entry)(void *), Shared *shared)
{
  int n;
  pthread_t thread;

  n = pthread_create (&thread, NULL, entry, (void *)shared);
  if (n != 0) {
    perror ("pthread_create failed");
    exit (-1);
  }
  return thread;
}
```

The return type from `pthread_create` is `pthread_t`, which you can think of as a handle for the new thread. You shouldn't have to worry about the implementation of `pthread_t`, but you do have to know that it has the semantics of a primitive type[1]. That means that you can think of a thread handle as an

[1] Like an integer, for example, which is what a `pthread_t` is in all the implementations I know.

immutable value, so you can copy it or pass it by value without causing problems. I point this out now because it is not true for semaphores, which I will get to in a minute.

If `pthread_create` succeeds, it returns 0 and my function returns the handle of the new thread. If an error occurs, `pthread_create` returns an error code and my function prints an error message and exits.

The parameters of `pthread_create` take some explaining. Starting with the second, `Shared` is a user-defined structure that contains shared variables. The following `typedef` statement creates the new type:

Listing:

```
typedef struct {
  int counter;
} Shared;
```

In this case, the only shared variable is `counter`. `make_shared` allocates space for a `Shared` structure and initializes the contents:

Listing:

```
Shared *make_shared ()
{
  int i;
  Shared *shared = check_malloc (sizeof (Shared));
  shared->counter = 0;
  return shared;
}
```

Now that we have a shared data structure, let's get back to `pthread_create`. The first parameter is a pointer to a function that takes a `void` pointer and returns a `void` pointer. If the syntax for declaring this type makes your eyes bleed, you are not alone. Anyway, the purpose of this parameter is to specify the function where the execution of the new thread will begin. By convention, this function is named `entry`:

Listing:

```
void *entry (void *arg)
{
  Shared *shared = (Shared *) arg;
  child_code (shared);
  pthread_exit (NULL);
}
```

The parameter of `entry` has to be declared as a `void` pointer, but in this program we know that it is really a pointer to a `Shared` structure, so we can typecast it accordingly and then pass it along to `child_code`, which does the real work.

When `child_code` returns, we invoke `pthread_exit` which can be used to pass a value to any thread (usually the parent) that joins with this thread. In this case, the child has nothing to say, so we pass `NULL`.

## B.3 Joining threads

When one thread wants to wait for another thread to complete, it invokes `pthread_join`. Here is my wrapper for `pthread_join`:

Listing:

```
void join_thread (pthread_t thread)
{
  int ret = pthread_join (thread, NULL);
  if (ret == -1) {
    perror ("pthread_join failed");
    exit (-1);
  }
}
```

The parameter is the handle of the thread you want to wait for. All my function does is call `pthread_join` and check the result.

## B.4 Semaphores

The POSIX standard specifies an interface for semaphores. This interface is not part of Pthreads, but most UNIXes that implement Pthreads also provide semaphores. If you find yourself with Pthreads and without semaphores, you can make your own; see Section 9.2.

POSIX semaphores have type `sem_t`. You shouldn't have to know about the implementation of this type, but you do have to know that it has structure semantics, which means that if you assign it to a variable you are making a copy of the contents of a structure. Copying a semaphore is almost certainly a bad idea. In POSIX, the behavior of the copy is undefined.

In my programs, I use capital letters to denote types with structure semantics, and I always manipulate them with pointers. Fortunately, it is easy to put a wrapper around `sem_t` to make it behave like a proper object. Here is the `typedef` and the wrapper that creates and initializes semaphores:

Listing:

```
typedef sem_t Semaphore;

Semaphore *make_semaphore (int n)
{
  Semaphore *sem = check_malloc (sizeof(Semaphore));
  int ret = sem_init(sem, 0, n);
```

```
  if (ret == -1) {
    perror ("sem_init failed");
    exit (-1);
  }
  return sem;
}
```

`make_semaphore` takes the initial value of the semaphore as a parameter. It allocates space for a `Semaphore`, initializes it, and returns a pointer to `Semaphore`.

`sem_init` uses old-style UNIX error reporting, which means that it returns -1 if something went wrong. Once nice thing about these wrapper functions is that we don't have to remember which functions use which reporting style.

With these definitions, we can write C code that almost looks like a real programming language:

Listing:

```
Semaphore *mutex = make_semaphore(1);
sem_wait(mutex);
sem_post(mutex);
```

Annoyingly, POSIX semaphores use `post` instead of `signal`, but we can fix that:

Listing:

```
int sem_signal(Semaphore *sem)
{
  return sem_post(sem);
}
```

That's enough cleanup for now.